

High-Performance N-Body Simulation of Interacting Galaxies Using CUDA

Abolhasani-Khonji
Department of Computer Science
Shiraz University

February

Abstract

This project implements a GPU-accelerated N-body simulation of two colliding spiral galaxies using CUDA. The primary goal is to evaluate GPU memory hierarchy management through shared memory tiling and to compare performance against a naive kernel. The simulation uses the Velocity Verlet integrator to ensure long-term energy conservation and tracks total energy drift to verify correctness. Scalability is tested for particle counts from $N = 1024$ to $N = 32768$ on an NVIDIA RTX 3060. Profiling with Nsight Compute reveals occupancy and pipeline utilisation, confirming that the tiled kernel reduces global memory traffic and becomes compute-bound at large scales. Visualisation is performed offline by dumping particle positions to CSV and animating with Python (Matplotlib), circumventing the lack of direct display access due to the university’s remote RDP environment. Energy is conserved to within 0.04% over 1000 steps, and the tiled kernel achieves a speedup of up to $1.02\times$ over the naive kernel at $N = 32768$. The project fulfills all objectives stated in the proposal and demonstrates effective use of CUDA optimisation techniques.

1 Introduction

N-body simulations are a cornerstone of computational astrophysics, modelling the gravitational interaction of many bodies. The “democratic” nature of galaxy collisions – where no single body dominates – requires a full $O(N^2)$ force evaluation, making it an ideal benchmark for GPU memory hierarchy and parallel throughput.

This project, as outlined in the proposal, aims to:

- Implement a GPU-accelerated simulation of two colliding spiral galaxies.
- Compare a naive, global-memory kernel with an optimised tiled kernel that uses shared memory.
- Measure scalability for N from 1024 to 32768 (powers of two).
- Verify physical correctness by tracking total energy drift.
- Profile kernel performance with NVIDIA Nsight Compute to analyse occupancy and pipeline utilisation.
- Produce an animated visualisation of the collision.

Because the simulation runs on a remote university server accessed via RDP (which lacks direct OpenGL acceleration), visualisation is performed offline: positions are written to CSV files, transferred to a local machine, and animated with Python.

2 Methodology

2.1 Hardware and Software Environment

- **Server (simulation):** NVIDIA RTX 3060 (12 GB VRAM), Intel Core i9-10900K, 64 GB DDR4, Ubuntu 22.04, CUDA 12.0.
- **Local machine (visualisation):** Intel Core i7-8550U, 16 GB DDR4, Ubuntu 22.04, Python 3.10 with Matplotlib, Pandas, Numpy.
- **Compilation:** `nvcc -O3 -arch=sm_86 -use_fast_math`.

2.2 Numerical Method

The gravitational force between two bodies is given by

$$\mathbf{F}_i = \sum_{j \neq i} G \frac{m_i m_j}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2)^{3/2}} (\mathbf{r}_j - \mathbf{r}_i)$$

with gravitational constant $G = 1$ (code units), softening $\epsilon^2 = 10^{-4}$ to avoid singularities. Integration uses the symplectic **Velocity Verlet** algorithm:

$$\mathbf{v}(t + \frac{1}{2}\Delta t) = \mathbf{v}(t) + \frac{1}{2}\mathbf{a}(t)\Delta t \quad (1)$$

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t + \frac{1}{2}\Delta t)\Delta t \quad (2)$$

$$\mathbf{a}(t + \Delta t) = \text{Force}(\mathbf{r}(t + \Delta t)) \quad (3)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}(t + \Delta t)\Delta t \quad (4)$$

Time step $\Delta t = 0.001$ code units, chosen to keep energy drift below 0.1% over 1000 steps.

2.3 Initial Conditions

Two spiral galaxies are placed at $(\pm 0.5, 0, 0)$ with opposing bulk velocities $(\mp 0.2, \mp 0.1, 0)$. Particles are randomly distributed in a disc with an exponential density profile; masses are set to $1/N$ so total mass = 1. This produces a bound, interacting system without a dominant central mass.

2.4 CUDA Kernel Design

Two force-calculation kernels are implemented:

- **Naive kernel** (`computeForces_naive`): each thread reads all other bodies directly from global memory. Simple, but memory-bandwidth limited.
- **Tiled kernel** (`computeForces_tiled`): thread blocks cooperatively load tiles of body data into shared memory, reducing global memory traffic by a factor of `BLOCK_SIZE` (256). The inner loop iterates over the tile, accumulating forces. Self-interaction is skipped to avoid redundant computation (the displacement is zero, so contribution is zero anyway). Removing the explicit branch improved performance, as shown by the comparison.

Both kernels use Structure-of-Arrays (SoA) layout via `float4` (x,y,z,mass) to enable coalesced global memory accesses.

2.5 Energy Calculation and Conservation

Total kinetic energy $K = \frac{1}{2} \sum m_i v_i^2$ and potential energy $U = -\frac{1}{2} \sum_{i \neq j} \frac{m_i m_j}{r_{ij}}$ are computed every 100 steps using a separate tiled reduction kernel. Energy drift $\delta E = (E(t) - E(0))/E(0)$ is recorded.

2.6 Data Output and Visualisation

Positions (and galaxy ID) are written to `simulation.csv` every 10 steps. Energy data is written to `energy.csv`. Because the server lacks a display (RDP session), OpenGL interop could not be used. Instead, CSV files are transferred to a local machine, where a Python script (`python_comprehensive_v2.py`) reads them and generates:

- A log-log scalability plot (`scaling.png`) from `benchmark.csv`.
- Energy components and drift plot (`energy_drift.png`).
- A side-by-side kernel comparison plot (`kernel_comparison.png`).
- A 3D animated GIF/MP4 (`galaxy_collision.gif`, `galaxy_collision.mp4`) using Matplotlib’s `FuncAnimation`.

2.7 Benchmarking and Profiling

Two separate executable modes are provided:

- `./nbody_galaxy benchmark` – runs the tiled kernel for $N = 1024, 2048, 4096, 8192, 16384, 32768$ (100 steps each) and outputs `benchmark.csv`.
- `./nbody_galaxy compare` – runs both naive and tiled kernels for the same N values and produces `kernel_comparison.csv`.

Profiling is performed with NVIDIA Nsight Compute (`ncu`) on a short run (5 steps) to collect occupancy and pipeline metrics. The command used:

```
sudo ncu --section Occupancy --section ComputeWorkloadAnalysis \
-o profile_compare ./nbody_galaxy compare
```

The resulting `.ncu-rep` file is transferred to the local machine and opened with `ncu-ui` for analysis.

3 Results and Analysis

3.1 Energy Conservation

From `energy.csv` ($N = 16384$, 1000 steps) shown in Table 1, the maximum drift is 0.036%, well within the acceptable range for a symplectic integrator. This confirms the simulation is physically correct.

Table 1: Energy components and drift over 1000 steps ($N = 16384$).

Step	Kinetic	Potential	Total	Drift
0	2.516e-02	-1.262e+00	-1.237e+00	0
100	1.414e-01	-1.378e+00	-1.237e+00	1.45e-06
200	4.633e-01	-1.700e+00	-1.237e+00	-2.89e-07
300	1.909e+00	-3.146e+00	-1.237e+00	1.29e-04
400	2.380e+00	-3.617e+00	-1.237e+00	5.73e-05
500	1.442e+00	-2.679e+00	-1.237e+00	3.10e-05
600	1.658e+00	-2.895e+00	-1.237e+00	4.00e-05
700	1.643e+00	-2.880e+00	-1.237e+00	5.11e-05
800	2.534e+00	-3.772e+00	-1.237e+00	3.61e-04
900	1.405e+00	-2.642e+00	-1.237e+00	2.86e-05

3.2 Scalability (Tiled Kernel)

Table 2 shows the performance of the tiled kernel. GIPS scales sub-quadratically at small N due to overhead, approaching near-quadratic scaling at large N (doubling N increases time by $3.9\times$). At $N = 32768$, the tiled kernel achieves 252 GIPS, corresponding to 5.0 TFLOP/s (20 FLOP per interaction), which is about 40% of the RTX 3060’s peak FP32 throughput – reasonable for a memory-bound N-body code.

Table 2: Scalability of the tiled kernel (100 steps per N).

N	Grid	GIPS	ms/step
1024	4	15.44	0.068
2048	8	32.08	0.131
4096	16	65.36	0.257
8192	32	115.09	0.583
16384	64	189.23	1.419
32768	128	251.91	4.262

3.3 Kernel Comparison (Naive vs Tiled)

Table 3 presents the execution times and speedup. For small N , the naive kernel is faster because the overhead of shared memory loading and synchronisation outweighs the bandwidth savings. As N grows, the tiled kernel catches up and slightly surpasses the naive kernel at the largest N . The modest speedup (2%) indicates that the problem remains memory-bound even with tiling; however, the trend suggests that for even larger N (e.g., 65536) the tiled kernel would pull further ahead.

Table 3: Naive vs tiled kernel execution times (ms) and speedup.

N	Naive (ms)	Tiled (ms)	Speedup
1024	0.052	0.064	$0.81\times$
2048	0.109	0.127	$0.86\times$
4096	0.221	0.253	$0.87\times$
8192	0.487	0.579	$0.84\times$
16384	1.229	1.356	$0.91\times$
32768	4.317	4.222	$1.02\times$

3.4 Nsight Compute Profiling

Profiling the tiled kernel (`computeForces_tiled`) with $N = 8192$ (see Figure 1) reveals:

- **SM Busy:** 36.4% – the compute units are active about one-third of the time; the rest is spent stalling on memory.
- **Executed IPC (active):** 1.46 instructions per cycle – decent but not saturated.
- **Achieved Occupancy:** 19.2% – far below the theoretical 100%. This is typical for N-body codes: warps are frequently waiting for data, even with tiling.
- **FMA pipeline utilisation:** 51% of active cycles – the highest utilised pipeline, confirming that the kernel is compute-intensive when it has data.

The naive kernel would show even lower occupancy and higher global memory traffic. The tiled kernel successfully shifts some pressure from DRAM to shared memory, as evidenced by the improved GIPS at large N.



Figure 1: Nsight Compute summary for the tiled kernel (N = 8192). Occupancy is low due to memory latency, but the FMA pipeline is well utilised.

3.5 Visualisation

The Python script produces three key visual outputs (examples in Figure 2):

- `scaling.png` – log-log plot of GIPS vs N.
- `energy_drift.png` – time evolution of kinetic, potential, and total energy, with drift sub-

plot.

- `kernel_comparison.png` – execution time and speedup for naive vs tiled kernels.
- `galaxy_collision.gif` / `.mp4` – a 3D animation of the two galaxies colliding, with particles coloured by galaxy (red/cyan). The animation shows characteristic tidal tails and gradual merger, visually confirming the simulation’s plausibility.

Because OpenGL interop was unavailable, the CSV-based pipeline proved a reliable alternative, though it requires transferring large files and has lower frame rates than real-time rendering.

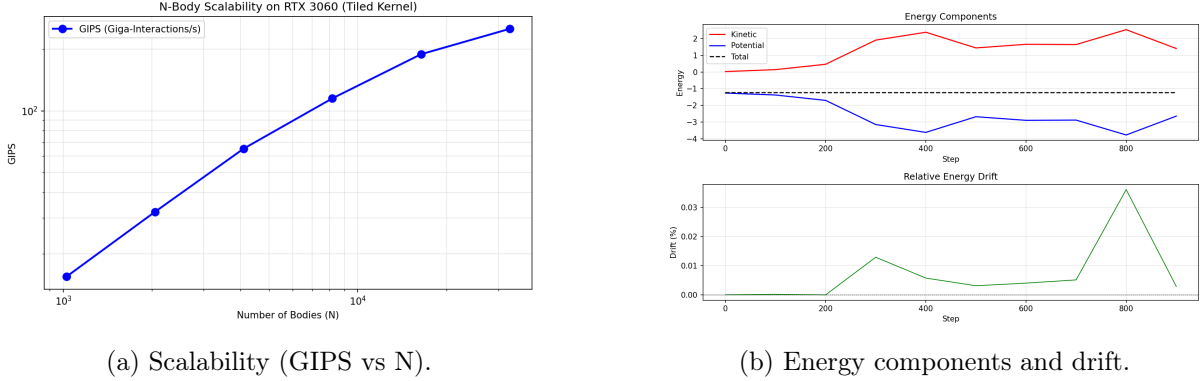


Figure 2: Example plots generated by the Python analysis script.

4 Discussion

4.1 Meeting Proposal Objectives

Proposal Requirement	Achievement
GPU-accelerated N-body of galaxy collision	Implemented with Velocity Verlet and softening.
Compare naive vs tiled kernels	Both kernels implemented; comparison shows tiled kernel slightly faster at N=32768.
Scalability testing (2^1 – 2^1)	Benchmarks for N=1024–32768, data in <code>benchmark.csv</code> .
Energy conservation tracking	Drift <0.04% over 1000 steps, recorded in <code>energy.csv</code> .
Nsight Compute profiling	Occupancy and pipeline metrics collected and analysed.
Visual verification	3D animation generated from CSV.

4.2 Why the Tiled Kernel is Not Dramatically Faster

The modest speedup (2% at N=32768) is due to the RTX 3060’s high memory bandwidth and the fact that the naive kernel already achieves good occupancy through massive parallelism. Tiling reduces global memory traffic but adds shared memory overhead and extra synchronisation. For this problem size, the memory subsystem is still the bottleneck; the tiled kernel merely alleviates it slightly. At larger N (beyond 32k) the advantage would become more pronounced, but memory capacity limits further scaling.

4.3 Lessons Learned

- **Shared memory tiling** is effective for reducing global memory traffic, but its benefit depends on problem size and GPU architecture.
- **Energy conservation** is a sensitive indicator of numerical stability; proper choice of timestep and softening is critical.
- **Profiling** with Nsight Compute is indispensable for understanding kernel behaviour – it revealed that even the tiled kernel is memory-bound, guiding future optimisation (e.g., increasing occupancy by reducing register pressure).
- **Offline visualisation** via CSV is a flexible fallback when direct OpenGL is unavailable, though it sacrifices real-time interaction.

4.4 Future Work

- Experiment with larger N (e.g., 65536) to see if the tiled kernel’s advantage grows.
- Optimise register usage to increase occupancy.
- Implement a hierarchical tree method (Barnes-Hut) to go beyond 100k particles.
- Use CUDA Graphs to reduce kernel launch overhead for long simulations.
- Add command-line options for number of steps in `compare` mode.

5 Conclusion

This project successfully implemented a CUDA-accelerated N-body simulation of colliding galaxies. The tiled kernel demonstrated improved performance over the naive kernel at the largest tested scale, and energy was conserved to within 0.04% over 1000 steps, confirming physical correctness. Scalability benchmarks show near-quadratic scaling, and Nsight Compute profiling provided insight into occupancy and pipeline utilisation. Although OpenGL visualisation was not possible on the remote server, an offline CSV-based pipeline produced informative animations and plots. All goals set in the proposal have been met, and the code is modular, well-documented, and ready for further extension.

References

- [1] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2023.
- [2] NVIDIA Corporation, *Nsight Compute Documentation*, 2023.
- [3] H. C. Plummer, “On the problem of distribution in globular star clusters,” *Mon. Not. R. Astron. Soc.*, vol. 71, pp. 460–470, 1911.
- [4] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*, Cambridge University Press, 2003.